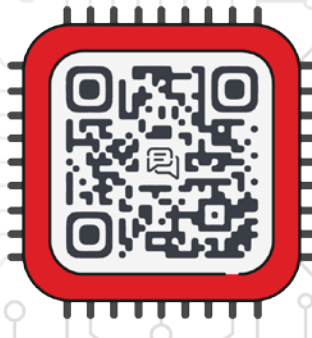




عملي

7

28

4200 sp

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

software testing types &
structural testing
techniques
عملي مشترك



RB Informatics; 09/06/2026

محتوى مجاني غير مخصص للبيع التجاري

هندسة برمجيات ٢

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

في المحاضرة السابقة تناولنا المفاهيم الأساسية لاختبار البرمجيات software testing وتعرفنا على الدور الحقيقي لمختبر البرمجيات software tester الذي لا يقتصر عمله على تجربة التطبيق واكتشاف الأخطاء الظاهرة، بل يمتد إلى فهم النظام بصورة شاملة، بدءاً من متطلبات العميل، وصولاً إلى التنفيذ البرمجي النهائي، كما ناقشنا العلاقة بين المتطلبات requirements وحالات الاستخدام use cases وحالات الاختبار test cases.

بعد أن أصبح لدينا فهم واضح لمفهوم الاختبار وأهميته، ولآلية بناء حالات الاختبار، نتقل في هذه المحاضرة إلى مرحلة أكثر تخصصاً تتمثل في دراسة أنواع اختبارات البرمجيات (Software Testing Types) والتصنيفات المختلفة المستخدمة في المشاريع البرمجية الحديثة.

استعدوا يا أصدقائي للمحاضرة الأخيرة من هذا المقرر!

في محاضرتنا الأخيرة من مقرر هندسة 2 ستكون محاورنا عن:

- 1- التمييز بين الاختبار اليدوي والاختبار المؤتمت.
- 2- المقارنة بين black box, white box, gray box.
- 3- مستويات الاختبار المختلفة.
- 4- فهم اختبارات القبول وأنواعها.
- 5- التمييز بين smoke testing, sanity testing, regression testing.
- 6- التعرف على اختبارات الجودة غير الوظيفية.
- 7- رسم مخطط تدفق التحكم (CFG).
- 8- أنواع التغطية.
- 9- حساب التعقيد الحلقي cyclomatic complexity بالطرق الـ 3.
- 10- استخراج المسارات المستقلة وبناء حالات الاختبار باستخدام path testing.



أولاً: أساليب تنفيذ الاختبارات:

تنقسم طرق تنفيذ الاختبارات إلى نوعين رئيسيين:

1. الاختبار اليدوي manual testing:

هو الاختبار الذي يقوم فيه المختبر بتنفيذ حالات الاختبار بنفسه بدون أدوات آلية، حيث يشغل النظام، يدخل البيانات، ينفذ السيناريوهات، ويقارن النتائج الفعلية بالنتائج المتوقعة بنفسه.

عيوبه:

- يحتاج وقتاً وجهداً كبيرين.
- عرضة للأخطاء البشرية.
- يصعب تكراره باستمرار.



ميزاته:

- مناسب لاختبارات تجربة المستخدم.
- مرونة عالية.
- إمكانية اكتشاف سلوكيات غير متوقعة.

2. الاختبار المؤتمت automated testing:

يتم فيه استخدام برامج أو أدوات تنفذ الاختبارات تلقائياً. أمثلة على الأدوات:

- Selenium
- Cypress
- Playwright
- Junit
- PHPUnit
- Jest

عيوبه:

- يحتاج وقتاً لإنشاء الاختبارات.
- يتطلب خبرة تقنية أعلى.



مميزاته:

- سرعة التنفيذ.
- سهولة التكرار.
- مناسب لاختبارات الانحدار regression test.

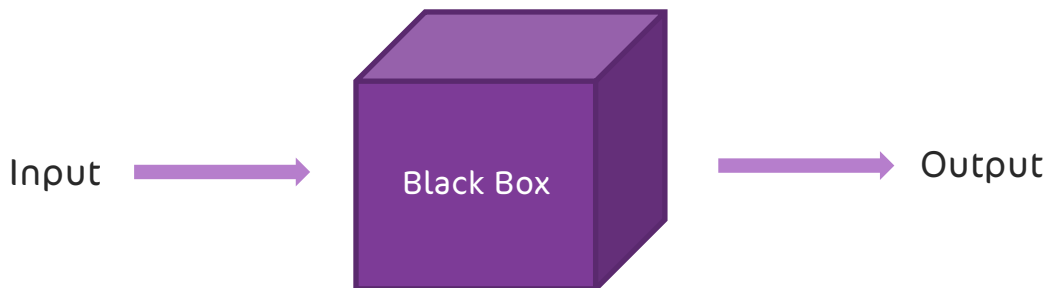
ثانياً: تصنيف الاختبارات حسب معرفة المختبر بالنظام:

1. Black Box Testing:

يعرف أيضاً باسم :

- Functional Testing
- Behavioral Testing

في هذا النوع لا يهتم المختبر ببنية النظام الداخلية، أو الكود البرمجي، يركز فقط على الدخل والخرج ويتحقق أن المخرجات صحيحة وفق المتطلبات.



مثال :

عند استخدامنا لمحرك البحث:

- يكتب المستخدم كلمة في مربع البحث.
- تظهر النتائج.

■ إذا كانت النتائج صحيحة، فإن الاختبار يعتبر ناجحاً.

المستخدم هو أكثر Black Box tester لا يهتم بالخوارزميات، أو قواعد البيانات المستخدمة.

خصائصه:

- لا يحتاج معرفة بالكود.
- يعتمد على المتطلبات.
- يحاكي سلوك المستخدم النهائي.
- يستخدم بشكل كبير في "اختبارات القبول".

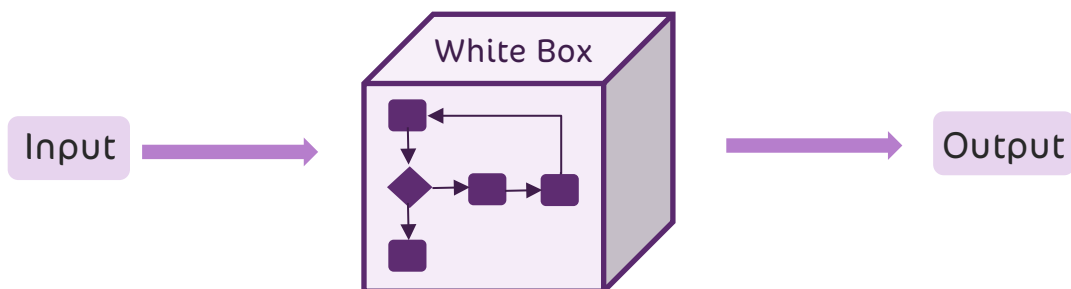
2. White Box Testing :

يعرف أيضا باسم:

- Structural Testing
- Glass Box Testing
- Clear Box Testing

في هذا النوع يمتلك المخبتر معرفة كاملة بالبنية الداخلية للنظام، حيث يتم التركيز على:

- الجمل البرمجية.
- الفروع الشرطية.
- الحلقات التكرارية.
- مسارات التنفيذ.



أهدافه:

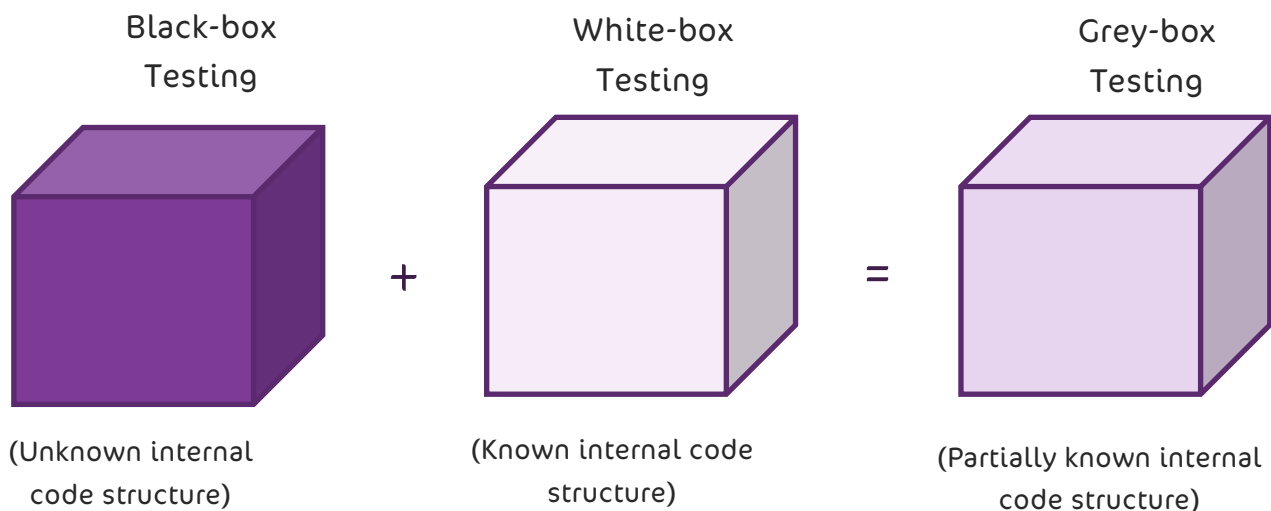
- ضمان تنفيذ جميع المسارات المهمة.
- اكتشاف الأخطاء المنطقية.
- قياس تغطية الكود Code Coverage.

(إذ إن الفحص عبر الصندوق الأسود قد يتسبب في تكرار الاختبارات بشكل عشوائي، فبينما يتسم الصندوق الأبيض بمسارات منظمة ومتسلسلة، قد يقوم الصندوق الأسود بتنفيذ اختبارات متتالية دون معرفة أنها متطابقة وتؤدي إلى النتيجة نفسها، مما ينتج عنه تكرار غير ضروري).

Black Box	White Box	
غير مطلوبة	مطلوبة	معرفة الكود
Functions الـ	البنية الداخلية	التركيز
المختبر أو المستخدم	المطور أو المختبر التقني	المنفذ
سلوكية	هيكلية	التغطية
أقل	أعلى	الصعوبة

3. Grey Box Testing:

يمثل حالة وسيطة بين النوعين السابقين.
يمتلك المختبر معرفة جزئية بالنظام لكنه يختبره من منظور المستخدم أيضا.



ثالثاً: مستويات الاختبار Testing Levels:

تتم عمليات الاختبار على عدة مستويات متدرجة:

1. Unit Testing:

أصغر مستويات الاختبار، يتم فيه اختبار أصغر وحدة مستقلة داخل البرنامج، مثل:

- Function
- Method
- Class

حيث يتم اختبار الدالة بمعزل عن بقية النظام.

فوائده :

- اكتشاف الأخطاء مبكراً.
- تسهيل الصيانة.
- زيادة جودة الكود.

2. Integration Testing:

بعد اختبار الوحدات بشكل منفصل، يتم اختبار تفاعلها مع بعضها.
"هل تعمل الوحدات المختلفة معاً بصورة صحيحة؟"

استراتيجيات اختبار التكامل:

3

Big Bang
Integration

2

Bottom-Up
Integration

1

Top-Down
Integration

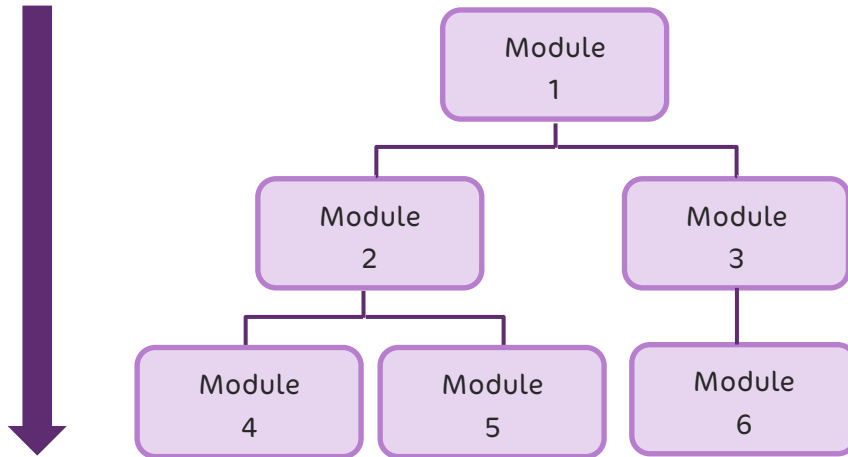
تخيل أن لديك برنامجاً مكوناً من 3 أجزاء:

- واجهة المستخدم (UI)
- خدمة الطلبات (Order Service)
- قاعدة البيانات (Database)

في Top-Down integration test :

نبدأ باختبار واجهة المستخدم أولاً، حتى لو لم تكن خدمة الطلبات أو قاعدة البيانات جاهزة بعد.

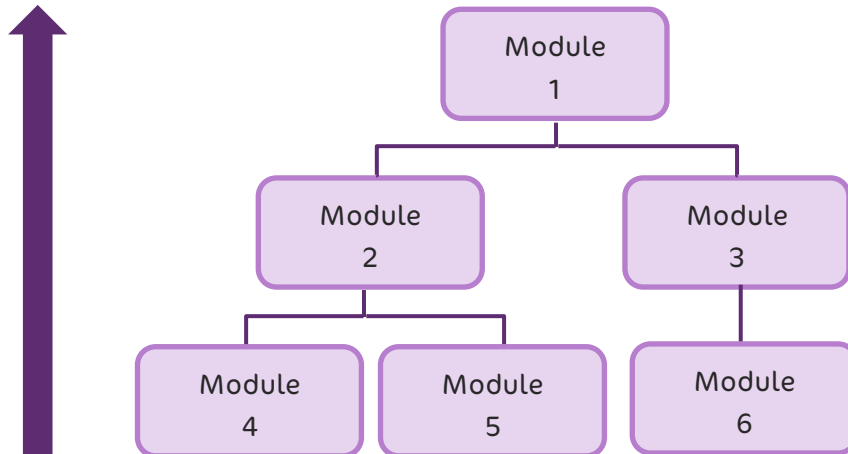
Top Down



في Bottom-Up Integration :

نبدأ باختبار قاعدة البيانات أولاً، ثم نضيف خدمة الطلبات، ثم واجهة المستخدم.

Bottom Up



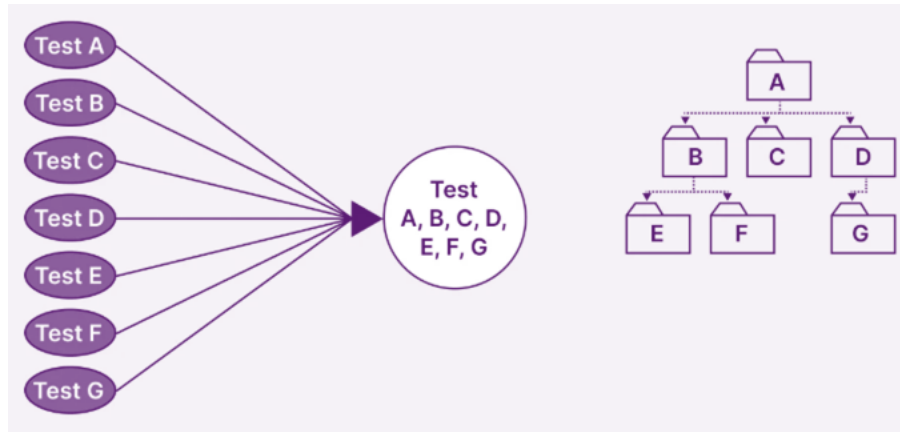
في Big Bang Integration :

ننتظر حتى تكتمل جميع أجزاء البرنامج، ثم نربطها كلها معاً ونبدأ الاختبار.

نختبر النظام كاملاً مرة واحدة.

المشكلة:

إذا ظهر خطأ، سيكون من الصعب معرفة هل السبب من الـ UI أم الـ Service أم الـ Database .



3. System Testing

اختبار النظام كاملاً، كوحدة متكاملة.

يتم التحقق من:

- الوظائف.
- الأداء.
- التوافق مع المتطلبات.

4. Acceptance Testing

يهدف الى التأكد من أن النظام يحقق احتياجات العميل, أنواعه :

1. (UAT) User Acceptance Testing

يقوم به المستخدم النهائي.
نركز هنا على:
سهولة الاستخدام، والتلبية الاحتياجات الفعلية.

1

2. Business Acceptance Testing (BAT)

ينفذه أصحاب المصلحة.
يهدف إلى التأكد من أن النظام يحقق
متطلبات الأعمال (Business Requirements).

2

3. Alpha Testing

يتم داخل الشركة المطورة قبل إطلاق
البرنامج، ويعد آخر اختبار داخلي قبل نشر
النظام.

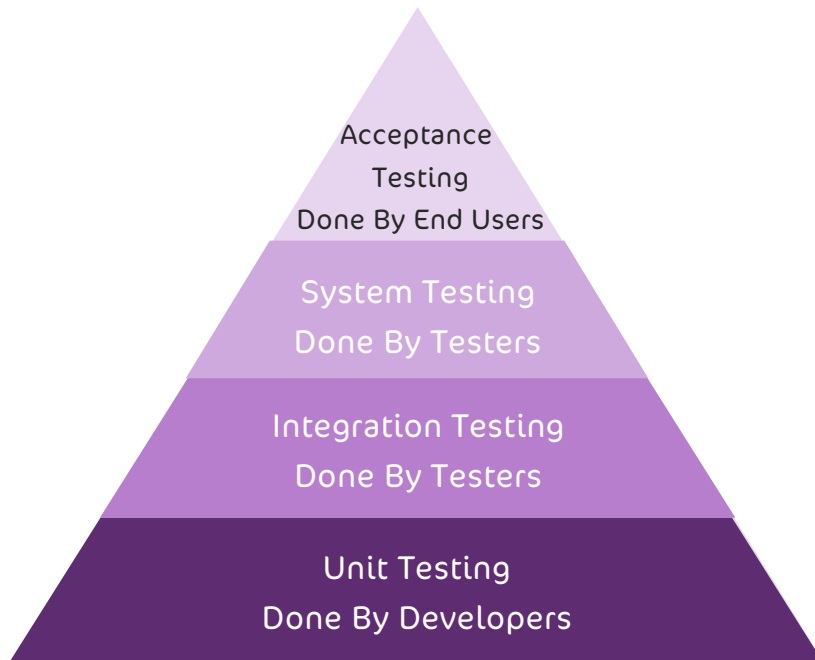
3

4. Beta Testing

يتم بعد نشر نسخة تجريبية من النظام
لمجموعة من المستخدمين الحقيقيين.
بهدف: جمع الملاحظات، اكتشاف المشكلات
الواقعية، تحسين المنتج قبل الإطلاق
النهائي.

4

Levels Of Testing:



رابعاً: اختبارات الصيانة والتحقق:

1. Regression Testing:

ينفذ بعد إصلاح خطأ أو إضافة ميزة جديدة، أو تعديل على النظام.
"هل أدى التعديل الجديد إلى كسر وظائف كانت تعمل سابقاً؟"
لهذا السبب يتم إعادة تشغيل مجموعة معينة من الاختبارات السابقة للتأكد من عدم ظهور آثار جانبية غير متوقعة.

2. Smoke Testing:

يعد اختباراً سريعاً، يهدف إلى التأكد من أن الوظائف الرئيسية للنظام تعمل بشكل أساسي.
أصل التسمية يعود إلى عالم الإلكترونيات، عند تشغيل جهاز إلكتروني للمرة الأولى إذا لم يخرج منه دخان، فهذا مؤشر أولي أنه يعمل، وبالمثل إذا اجتاز النظام الوظائف الأساسية بدون انهيار، فإنه يجتاز الـ smoke testing.

3. Sanity Regression:

اختبار محدود النطاق يتم بعد تعديل صغير أو إصلاح bug معين بهدف التأكد من أن الإصلاح نجح فعلاً.

• الفرق بينه وبين الـ regression testing :

Sanity Testing	Regression Testing
محدود.	واسع النطاق.
يركز على جزء محدد.	يشمل عدة أجزاء.
أسرع.	يستغرق وقتاً أكبر.
يستخدم بعد إصلاحات بسيطة.	يستخدم بعد تغييرات كبيرة.

خامساً: الاختبارات غير الوظيفية Non-Functional Testing:

بعد التأكد من أن الوظائف تعمل، نبدأ بدراسة جودة النظام نفسه.

1. Performance Testing :

يقيس سرعة واستقرار النظام.

وينقسم إلى:

:Load testing

اختبار الأداء تحت الحمل المتوقع، مثلاً: 500 مستخدم متصل بنفس الوقت.

:Stress testing

اختبار النظام تحت أحمال تتجاوز الحمل المتوقع، مثال: 10 آلاف مستخدم متزامن.

:Spike testing

اختبار الارتفاع المفاجئ في عدد المستخدمين أو الطلبات، مثلاً: من 100 مستخدم إلى 20 ألف مستخدم خلال دقائق، كما يحدث أثناء العروض أو الأحداث الكبرى .

أن يتوّج مسعانا بالوصول وألاً يخيب لنا أمل أبداً..
أمين..



2. Security Testing

يهدف إلى اكتشاف الثغرات الأمنية، يشمل:

- Authentication testing
- Authorization Testing
- Session Management
- Data Protection



إضافة من الأستاذ أنور:

SQL Injection

من أشهر الهجمات القديمة.

حيث يقوم المهاجم بإدخال أوامر SQL داخل حقول الإدخال.

أما اليوم فقد أصبحت أطر العمل الحديثة وأنظمة ORM تقلل بشكل كبير من هذا النوع من الثغرات. لكن الاختبارات الأمنية الحديثة أصبحت أكثر تعقيداً وتشمل مئات السيناريوهات الأخرى.

3. Usability Testing

يهتم بقياس : سهولة الاستخدام، وضوح الواجهات، تجربة المستخدم.
قد يكون النظام صحيحاً تقنياً لكنه معقد وصعب الاستخدام وهنا يفشل اختبار قابلية الاستخدام.

1. Compatibility Testing

يهدف الى التأكد من عمل الموقع على: مختلف الصفحات، مختلف أنظمة التشغيل، الأجهزة، أحجام الشاشات.
مثلاً: يجب التأكد من عمل الموقع على: safari, chrome, internet explorer.
إذ قد تعمل بعض التقنيات في متصفح معين وتفشل في متصفح آخر.

Functional Testing

Unit
TestingIntegration
TestingSystem
TestingAcceptance
Testing

Non-Functional Testing

Performance
TestingSecurity
TestingUsability
TestingCompatibility
Testing

سابعاً: White Box Testing ومخططات التدفق:

في الـ White Box Testing, يتم تحليل مسارات التنفيذ باستخدام:

- CFG (control flow Graph)
- Loop testing
- Path testing

مخطط تدفق التحكم (CFG) Control Flow Graph:

هو تمثيل رسومي يوضح كيفية انتقال التنفيذ بين أجزاء الكود.

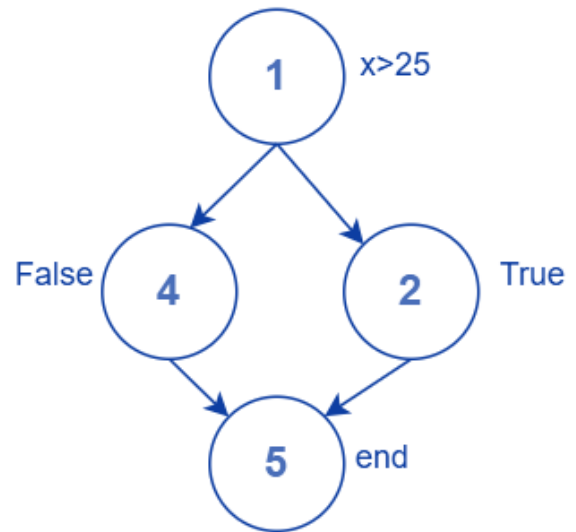
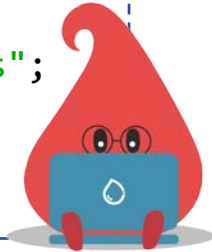
مكونات Control Flow Graph:

- العقدة (Node): تمثل مجموعة تعليمات تنفذ بشكل متتابع.
- الحافة (Edge): تمثل انتقال التحكم بين عقدتين.
- العقدة الشرطية (Predicate Node): تمثل قراراً منطقياً ينتج عنه أكثر من مسار (true||false) .

```

1 if (x > 25)
2   cout<<"Stray Kids";
3 else
4   cout<<"Anime";

```



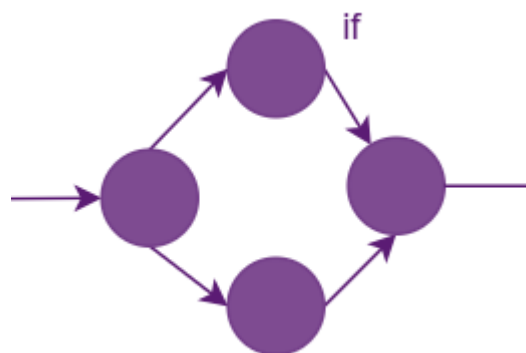
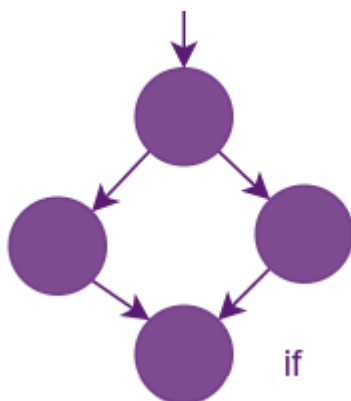
كيف نرسمه؟

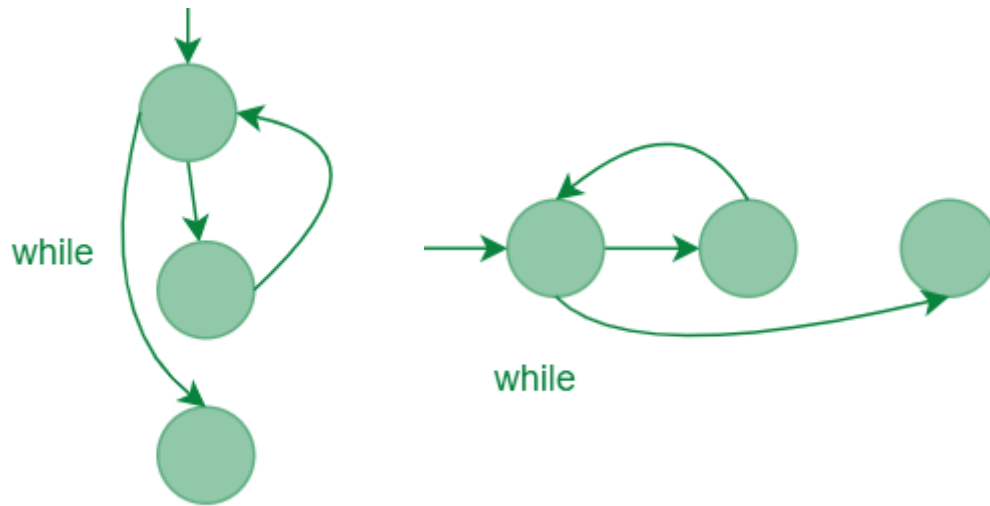
شكل الرسم:

sequence

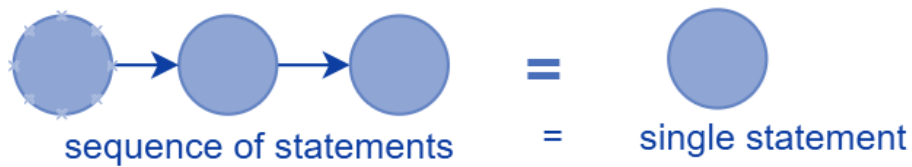


sequence





ملاحظة تبسيط الرسم:

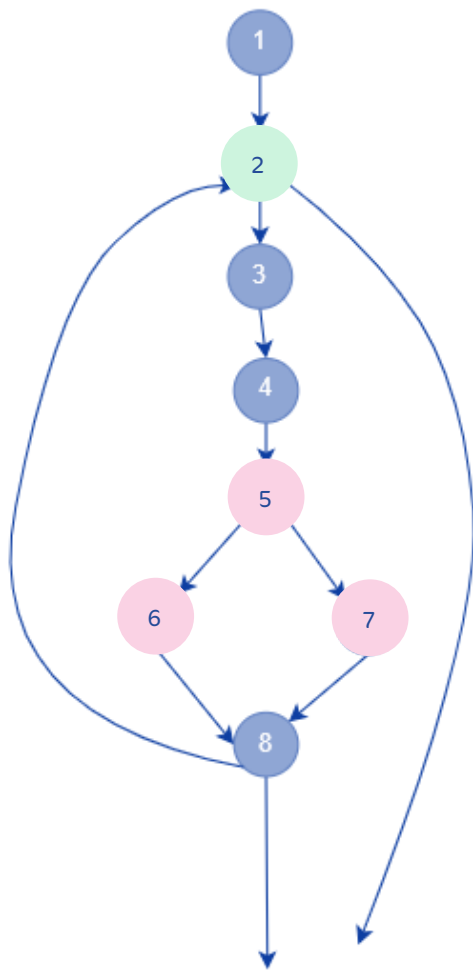


مثال :

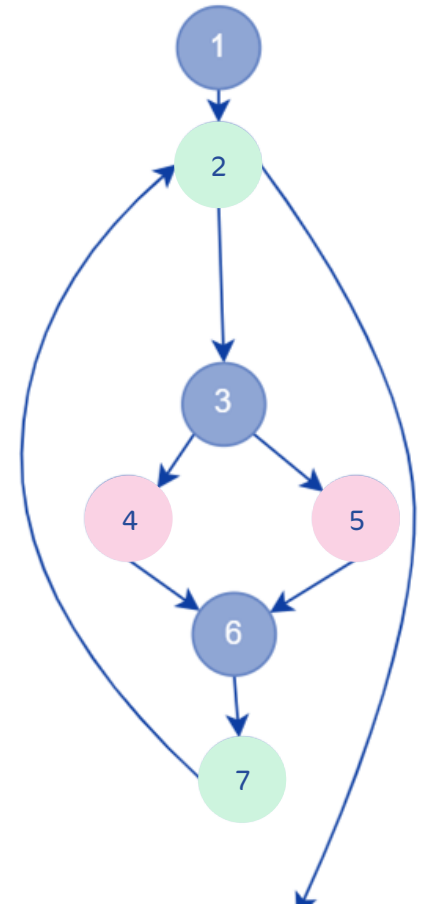
```

1 int s=0, d=0;
2 while (x<y){
3     x+=3;
4     y+=2;
5     if (x+y<100)
6         s+=y+x;
7     else d+=x-y;
8 }
    
```





لكن نريد تبسيطه، ولدينا تسلسل من التعليمات التي لا تعتبر قرار هنا، مثلاً 3 و4 نختصرهم بعقدة واحدة:



تمثل أرقام العقد في الصورة الأولى أرقام الأسطر في التعليمات البرمجية السابقة، بينما تمثل أرقام العقد في الصورة الثانية مجرد أرقام تسلسلية بعد الاختصار.

أنواع التغطية:

- Statement Coverage
- Branch Coverage
- path coverage



1. Statement Coverage

هدفها التأكد من أن كل تعليمة في الكود تم تنفيذها مرة واحدة على الأقل:

حسب المرسوم المرفق: نحتاج مسارين:

المسار الأول: 1,2,3,4,6,7

المسار الثاني: 1,2,3,5,6,7

2. Branch Coverage

يهدف للتأكد من أن كل فرع (if, else) تم تغطيته بكلتا حالتيه (True, false)

حسب الرسم الجديد التالي:

المسار الأول: 1,2,3,4,6,7,9

المسار الثاني: 1,2,3,5,6,8,9

المسار الثالث: 1,2,6,8,9

(غطينا كل التفرعات وليس كل المسارات).

3. Path Coverage

تغطية جميع المسارات المنطقية من البداية إلى النهاية:

المسار الأول: 1,2,3,4,6,8,9

المسار الثاني: 1,2,3,4,6,7,9

المسار الثالث: 1,2,3,5,6,8,9

المسار الرابع: 1,2,3,5,6,7,9

المسار الخامس: 1,2,6,8,9

المسار السادس: 1,2,6,7,9

ملاحظة: كل ما زاد عدد المسارات المغطاة زادة موثوقية الاختبار.

التعقيد الحلقي (Cyclomatic Complexity):

هو مقياس رياضي يستخدم لتحديد عدد المسارات المستقلة independent path داخل البرنامج، من خلال تحليل مخطط تدفق التحكم (CFG).

يعد هذا المقياس من أبرز مؤشرات تعقيد الكود، ويساعد في تحديد عدد حالات الاختبار اللازمة لتغطية جميع المسارات المنطقية في الكود.

كل مسار مستقل يتطلب حالة اختبار واحدة على الأقل لتغطيته، لذا هذا المقياس يساعدنا في تحديد الحد الأدنى لعدد

الاختبارات، وفهم مدى تعقيد الكود.

كلما زادت التعقيدات، زادت الحاجة إلى اختبارات أكثر شمولاً، وزادت صعوبة فهم الكود، وصيانتته.

الصيغ الثلاثة لحساب ال Cyclomatic Complexity:

الصيغة 1: عدد الشروط predicates:

$$CC = P + 1$$

P: عدد الشروط المنطقية (...، تفرعات , for , while , if)
يمكننا استخدامه عندما نقرأ الكود ونرصد الشروط يدوياً.

الصيغة 2: عناصر الرسم البياني (Nodes,Edges):

$$CC = E - N + 2$$

حيث:

E: عدد الوصلات – أي الأسهم التي هي انتقالات بين العقد.

N: عدد العقد

الصيغة 3: عدد المناطق المغلقة في الرسم:

$$CC = Bounded Areas + 1$$

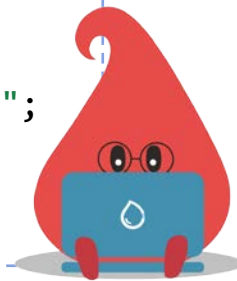
نستخدمه عند رسم مخطط التدفق Flow Chart حيث كل منطقة مغلقة تعتبر مسار مغلق.

إذاً، عندما يكون لدينا كود برمجي ونريد حساب تعقيده الحلقى، فإننا:

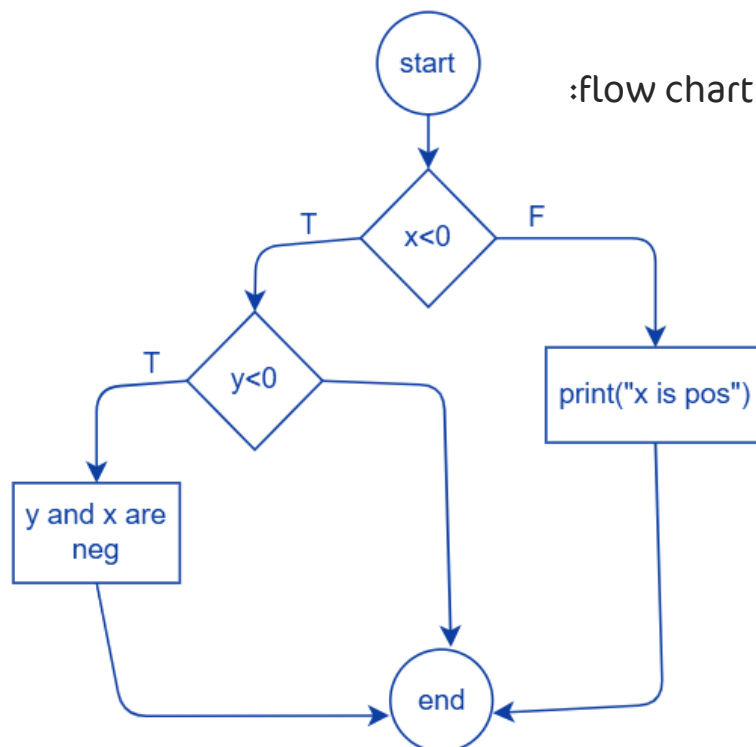
1. نحول الكود الى Flow Chart.
2. نحوله لرسم graph ونقوم بتبسيطه.
3. نحسب التعقيد الحلقى من ال graph المبسط.

لدينا الكود:

```
1 if (x<0)
2 {
3     if(y<0){
4         cout<<"Y and X are negative!";
5     }
6 }else
7     cout<<"x is positive here man!";
```

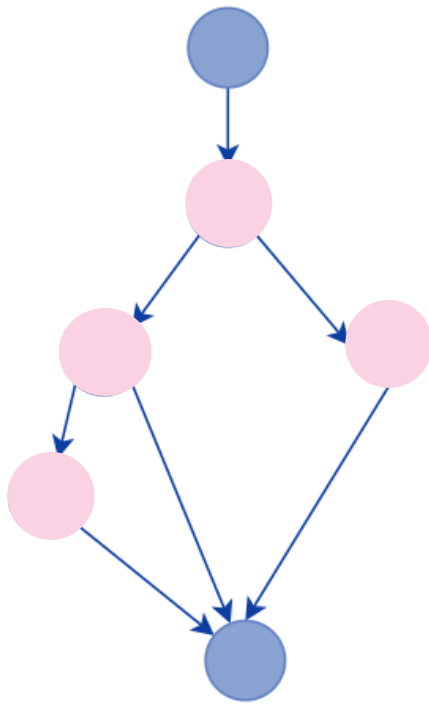


1. نقوم بتحويل الكود إلى flow chart:



2. نحول الـ flow chart إلى CFG:

تحليل كيف تم رسمه:



■ بداية لدينا نقاط أساسية :

لدينا في الكود شرط أول ($x < 0$)داخله شرط ثاني ($y < 0$)

إذا تحققاً معاً نطبع (كلاهما سالب)

إذا لم يتحقق الشرط الأول (نطبع ان العدد الأول موجب)

وفي جميع الحالات ينتقل البرنامج الى النهاية

■ ثم نحدد العقد:

كل نقطة منطقية تكون عقدة لذا لدينا 6 عقد:

start: بداية الدالة

if ($x < 0$) : الشرط الأولif ($y < 0$) : الشرط الثاني

cout << "poth are negative" : العملية في حال تحقق الشرطين

cout << "x is positive" : العملية في حال لم يتحقق الشرط الأول

end: نهاية الكود

بعدها نقوم بتحديد الوصلات بين العقد.

الآن أصبح بإمكاننا رؤية المسارات المستقلة في التنفيذ من خلال الرسم.

3. لنحسب التعقيد الحلقى بالطرق الـ 3:

● الطريقة الأولى: $cc = P + 1$

نحسب عدد الشروط (الموضحة باللون الأحمر في المخطط)

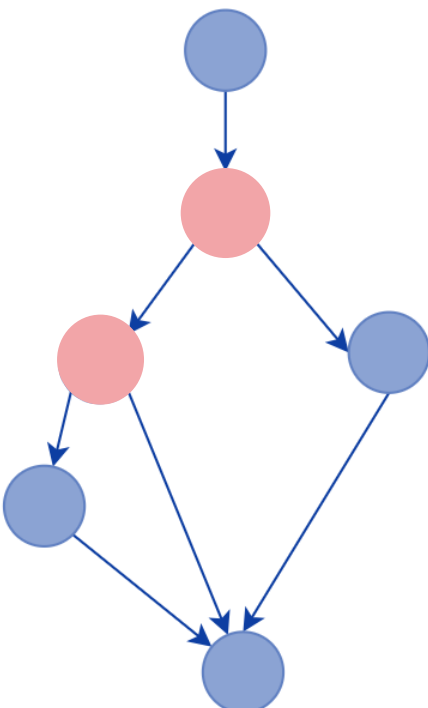
if ($x < 0$)if ($y < 0$)

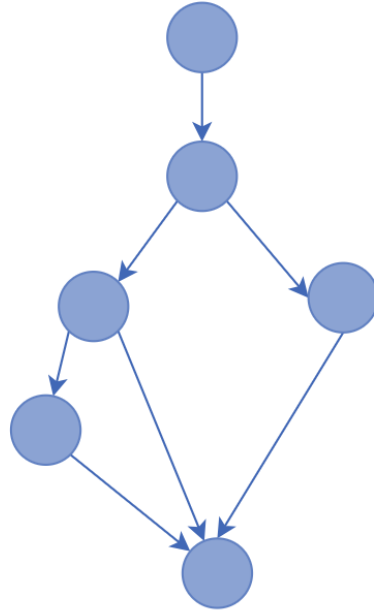
else لا يعتبر كشرط بل تابع لـ if

بالتالي :

 $cc = 2 + 1 = 3$

يوجد 3 مسارات مستقلة في هذا الكود.





● الطريقة الثانية : $cc=e-n+2$

نحسب عدد العقد $N=6$

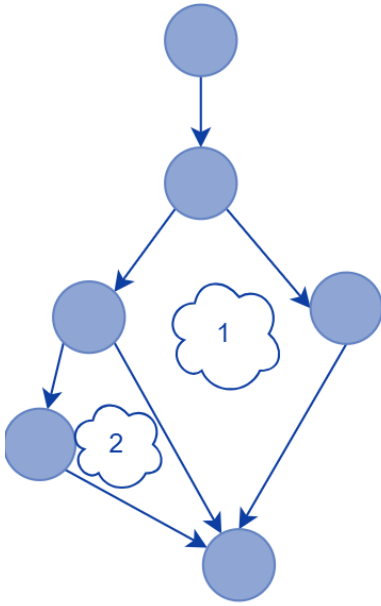
عدد الوصلات $E=7$

بالتالي:

$$cc=7-6+2$$

$$cc=3$$

النتيجة مطابقة للطريقة الأولى، 3 مسارات (منطقي)

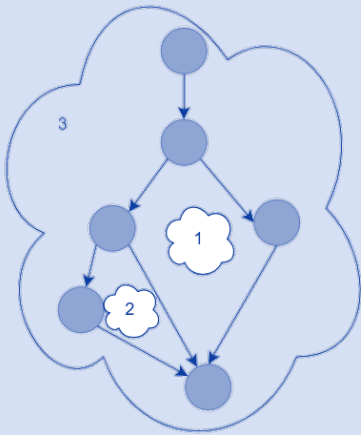


● الطريقة الثالثة : باستخدام المناطق المغلقة:

واضح من الرسم أنه لدينا منطقتان مغلقتان

$$cc=2+1=3$$

أيضاً نفس النتيجة (يجب أن تكون النتيجة نفسها بجميع الطرق).



ملاحظة للطريقة الثالثة:

أمكننا القول أن القانون يعتمد على (عدد المناطق المغلقة + 1)،

أو الاعتماد على المناطق المغلقة فقط، حيث يُعدُّ

الرسم بأكمله وفقاً للمخطط منطقة واحدة.

مثال على ذلك:

تؤدي هذه الطرق إلى النتيجة ذاتها.

إضاءة:

المسار المستقل independent path:

هو مسار يمر عبر حافلة جديدة لم يتم تغطيتها في المسارات السابقة،
إذا كانت $cc=4$ أي أنه يوجد 4 مسارات مستقلة.

خطوات path-based testing:

1. رسم cf و .cf

2. حساب cc.

3. استخراج المسارات الأساسية basis path set.

4. بناء حالة اختبار لكل مسار

مثال: تابع Average :

التابع يحسب متوسط القيم الموجودة داخل المصفوفة بشرط:

- أن تكون القيمة بين $\min$, $\max$.
- أن لا تصل إلى القيمة النهائية 999- .

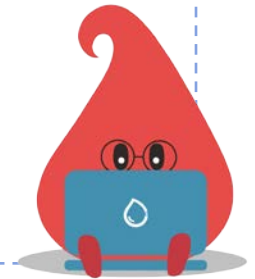
Take the RISK or Lose The
CHANCE



```

1 int average(int value[], int min, int max, int N)
2 {
3     int i, totalValid, sum, mean;
4
5     i = 0;
6     totalValid = 0;
7     sum = 0;
8
9     while (i < N && value[i] != -999)
10    {
11        if (value[i] >= min && value[i] <= max)
12        {
13            totalValid++;
14            sum += value[i];
15        }
16
17        i++;
18    }
19
20    if (totalValid > 0)
21        mean = sum / totalValid;
22    else
23        mean = -999;
24
25    return mean;
26 }

```



الخطوة الأولى: رسم المخطط:

1. نقوم بتحديد العقد:

الوصف	العقدة
Start	1
While	2
If condition	3
++ total valid	4
Sum+= val[i]	5

i++	6
Back to while	7
If (totalValid >0)	8
Mean= sum/ totalValid	9
Mean=-999	10
return	11

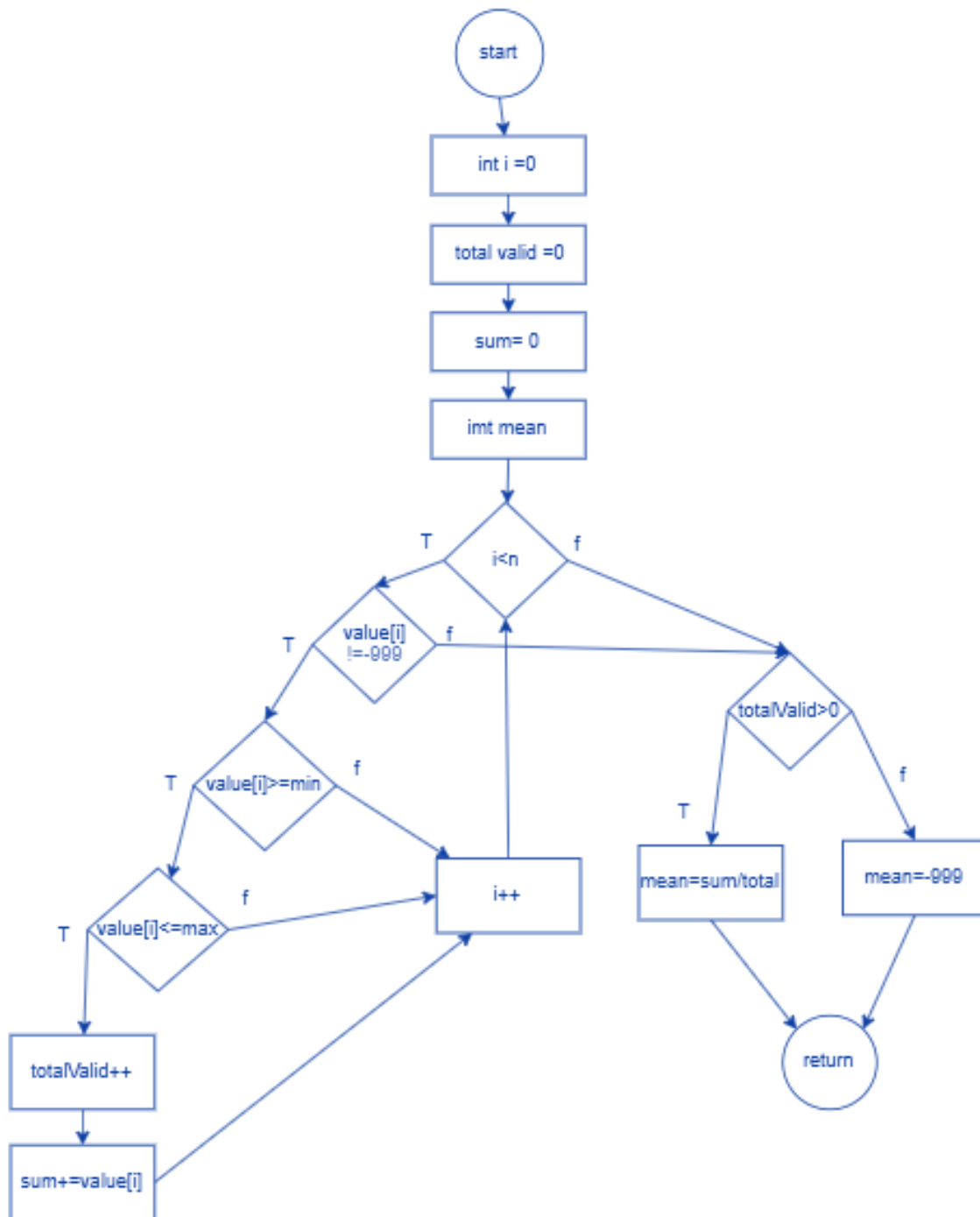
أي:

```

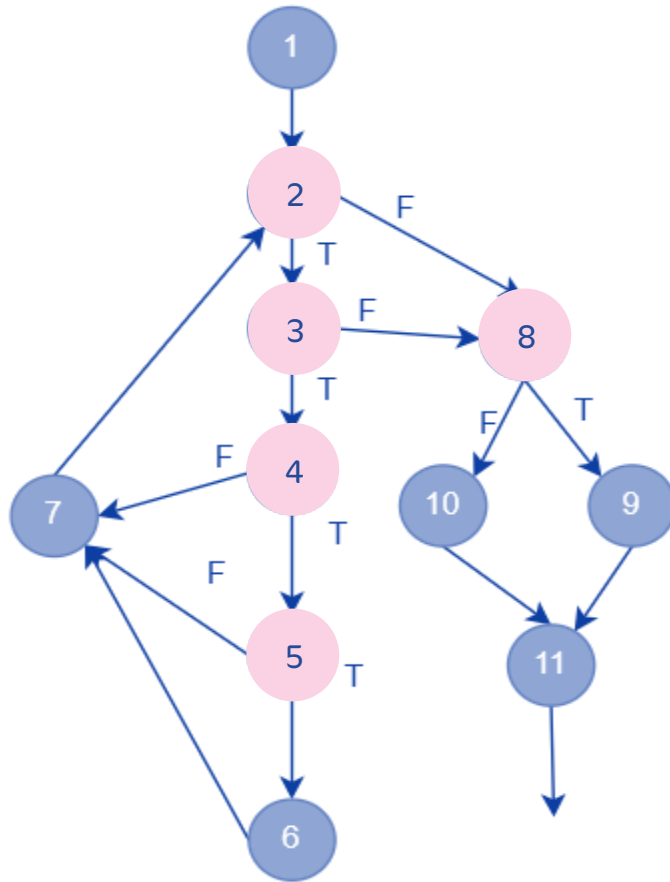
1  int average(int value[], int min, int max, int N)
2  {
3      int i, totalValid, sum, mean;
4
5      i = 0;
6      totalValid = 0;
7      sum = 0;
8
9      while (i < N && value[i] != -999)
10     {
11         if (value[i] >= min && value[i] <= max)
12         {
13             totalValid++;
14             sum += value[i];
15         }
16
17         i++;
18     }
19
20     if (totalValid > 0)
21         mean = sum / totalValid;
22     else
23         mean = -999;
24
25     return mean;
26 }

```

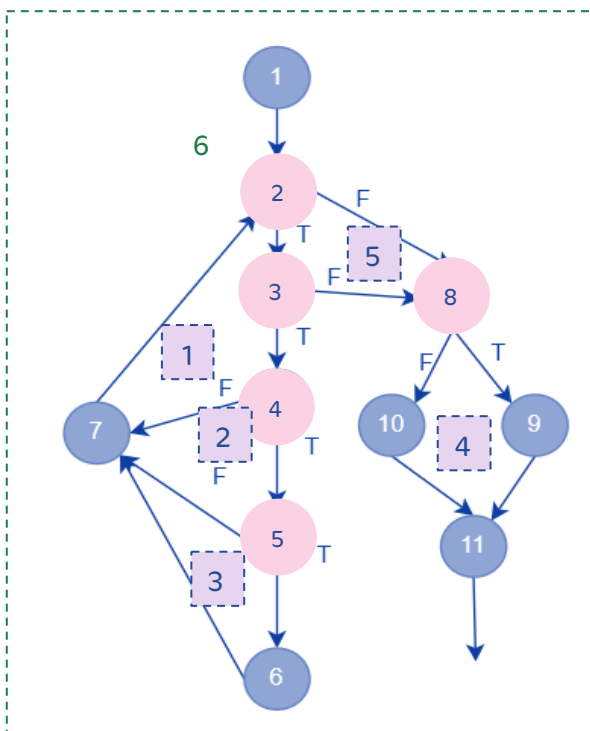




لنقل أن هذا الرسم التدفقي يسهل عملية التحويل إلى (CFG) ، دون الأخذ بعين الاعتبار حساب التعقيد هنا.



الخطوة الثانية: حساب التعقيد الحلقي:



نختار الطريقة التي نريدها:

حسب الطريقة الثالثة لدينا:

6 مناطق

CC= 6

حسب الطريقة الأولى:

$P+1$

العقد الوردية: هي عقد القرار كما نلاحظ عددها خمسة:

$$CC=5+1$$

cc=6

الخطوة الثالثة: استخراج المسارات المستقلة independent paths:

المسار الأول P1:

عدم دخول الحلقة نهائياً: 1,2,8,10,11

يتنفيذ عند حالة الاختبار: $n=0$

الناتج = -999

المسار الثاني P2:

أول عنصر = -999 : 1,2,3, 8,10,11

مثال: $value[] = \{-999\}$ $N=1$

الناتج: -999

المسار الثالث P3:

قيمة أصغر من min : 1,2,3,4,7,2,8,10,11

مثال: $value[] = \{25\}$ $N=1$ $min=30$

الناتج = -999

المسار الرابع P4:

قيمة أكبر من max : 1,2,3,4,5,7,2,8,10,11

مثال: $value[] = \{99\}$ $n=1$ $max=90$

بالتالي:

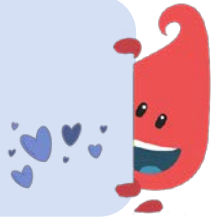
 $average = -999$

المسار الخامس P5:

قيمة صحيحة واحدة: 1,2,3,4,5,6,7,2,8,9,11

مثال: $value[] = \{25\}$ $N = 1$ $min = 0$

كلّ الصعاب بلطف الله هينةً
طمّن فؤادك لا حزن ولا قلق



max = 100

بالتالي :

sum = 25

الخرج المتوقع : average = 25

المسار السادس P6:

1,2,3,4,5,6,7,2,3,4,5,6,7,2,8,9,11

Test case:

value[]={25,75}

N=2

min=0

max=100

بالتالي:

الخرج المتوقع : average=50

الخطوة الرابعة: جدول حالات الاختبار:

Test Case	المدخلات	المسار	النتائج المتوقعة
TC1	N=0	P1	-999
TC2	Value={-999}	P2	-999
TC3	Value={25} min=30	P3	-999
TC4	Value={99} max=90	P4	-999
TC5	Value={25} min=0 max=100	P5	25
TC6	Value={25,75} min=0 max=100	P6	50

ملخص المحاضرة:

اختبار البرمجيات عملية أساسية لضمان جودة الأنظمة البرمجية، ويشمل مجموعة واسعة من الأنواع والمستويات. تبدأ العملية من اختبار الوحدات البرمجية الصغيرة، ثم تكاملها، ثم اختبار النظام كاملاً، وتنتهي باختبارات القبول. كما توجد اختبارات غير وظيفية مثل الأداء والأمان وقابلية الاستخدام.

أما في الاختبار الهيكلي White Box Testing فتستخدم أدوات مثل Cyclomatic و Control Flow Graph و Complexity لاكتشاف جميع مسارات التنفيذ وضمان تغطية الكود بصورة منهجية.

